Binaml: Continual Learning over Binary Feature Streams

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

Binaml focuses on continual learning models over streams of binary features subject to data drift. Binary features provide a task-agnostic representation and enable models optimized for memory, latency, and energy efficiency. This paper specifies Binaml's online ensemble models, `BRegressor` for scalar regression and `BClassifier` for multiclass classification, as ensembles of batch-learned conjunction experts with task-specific linear heads. Both models adapt online rather than relying on a stationary train/serve assumption and learn compact conjunction representations from residual feedback. Runtime memory is fixed at model construction and the online hot path performs zero heap allocation after `new`. The models are environment-agnostic: they do not depend on how the stream is produced. This paper specifies the current models and reports reproducible prequential comparisons on versioned synthetic regression and classification streams.

1 Notation

Symbols below are used consistently in this paper and in `crates/binaml-core`.

d, x_t Input width and binary input (`source_feature_count` / `n_features`).

$y_t, \hat{y}_t$ Scalar regression target and prediction.

$C, c_t, \hat{c}_t$ Number of classes, class label, and predicted class (`n_classes`).

F, F_t Current expert count and expert count after batch finalizations through time t (`function_count()`); $F \leq K_{\max}$.

e_t, s_t Residual and supervision label appended to the expert batch.

b, w_k, η, λ Intercept, ensemble weight of expert k, learning rate (`learning_rate`), and ℓ_2 decay (`l2`).

$f_k(x)$ Conjunction expert k: an AND of signed input literals, stored as a compact `ConjunctionExpert`.

$\ell_k, \ell_{\max}$ Literal count in expert k and maximum literals per conjunction (`max_conjunction_length`).

B, S, n Expert batch size (`batch_size`), SGD steps per observation (`sgd_steps`), and batch index.

$K_c, K_{\max}, L_s$ Beam width (`max_conjunctions`), ensemble capacity (`max_functions`), and stale-layer patience (`stale_layers`): stop extending after L_s consecutive depths without improving the best in-sample batch accuracy seen so far.

2 Model

Both `BRegressor` and `BClassifier` maintain an ensemble of conjunction experts. Expert structure evolves slowly through batch-local beam search over literal conjunctions and in-sample output selection; expert weights in the linear head adapt on every observation. At capacity $K_{\max}$, ensemble pruning drops the weakest expert. The two models differ only in the linear head that maps expert activations to predictions.

2.1 Regression head

At time t, the regressor receives $x_t \in \{0,1\}^d$ and later observes a finite scalar y_t. If useful signal is sparse weighted boolean structure over bits, a linear head over learned boolean indicators is a direct hypothesis class: continuous amplitudes, discrete experts, and an inspectable composition. Let F_t be the number of experts after batch finalizations through time t. The prediction is

$$\hat{y}_t = b + \sum_{k=1}^{F_t} w_k \, f_k(x_t),$$

where $b \in \mathbb{R}$, each $w_k \in \mathbb{R}$, and each $f_k(x_t) \in \{0,1\}$. Each f_k is a compact conjunction expert built from one supervision batch. Expert construction consumes a binary supervision bit derived from the scalar target. With pre-update prediction $\hat{y}_t$, the residual is $e_t = y_t - \hat{y}_t$ and the label appended to the current expert batch is

$$s_t = \mathbf{1}\{e_t \geq 0\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. For one SGD step with learning rate $\eta > 0$ and decay $\lambda \geq 0$, first apply ℓ_2 decay to every stored weight, then update the intercept and weights from the pre-step residual:

$$\begin{aligned} w_k &\leftarrow (1 - \eta\lambda)\, w_k && \text{for all stored } k, \\ b &\leftarrow b + \eta e_t, \\ w_k &\leftarrow w_k + \eta e_t f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

2.2 Classification head

The classifier receives $x_t \in \{0,1\}^d$ and later observes a class label $y_t \in \{0, \ldots, C-1\}$. Let F_t be the number of experts after batch finalizations through time t. For each class c, logits are

$$\ell_{t,c} = b_c + \sum_{k=1}^{F_t} w_{k,c} \, f_k(x_t),$$

where $b_c \in \mathbb{R}$ and each $w_{k,c} \in \mathbb{R}$. The predicted class is $\hat{c}_t = \operatorname{argmax}_c \ell_{t,c}$, with ties broken toward the smallest index. Expert construction consumes a binary supervision bit derived from the class label. With pre-update prediction $\hat{c}_t$, the label appended to the current expert batch is the misclassification indicator

$$s_t = \mathbf{1}\{\hat{c}_t \neq y_t\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. Let $p_{t,c} = \mathrm{softmax}(\ell_t)_c$ be the pre-step class probabilities. For one SGD step on softmax cross-entropy with learning rate $\eta > 0$ and decay $\lambda \geq 0$, define the class errors $\delta_{t,c} = p_{t,c} - \mathbf{1}\{c = y_t\}$. First apply ℓ_2 decay to every stored weight, then update intercepts and weights:

$$\begin{aligned} w_{k,c} &\leftarrow (1 - \eta\lambda)\, w_{k,c} && \text{for all stored } k, c, \\ b_c &\leftarrow b_c - \eta\, \delta_{t,c}, \\ w_{k,c} &\leftarrow w_{k,c} - \eta\, \delta_{t,c}\, f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

3 Building experts

Binaml builds and maintains a collection of conjunction experts. Each expert is an AND of signed input literals; the ensemble grows one expert per filled batch while weights w_k update on every observation. Expert structure therefore evolves slowly, on batch boundaries, while the linear head tracks the task quickly. Candidate conjunctions are grown one literal at a time and ranked by in-sample accuracy, with association score as a tie-break.

3.1 Expert representation and batch learning

Each expert batch runs a layered beam search. Depth 0 holds up to K_c non-constant signed literals (one literal per candidate). To extend depth d to depth $d{+}1$, the builder ANDs each depth d parent column with a new signed literal column drawn from depth 0, skipping literals already present in the parent key and discarding constant results. Candidates are deduplicated by a `ConjunctionKey` (positive and negative bit masks over input coordinates), pruned to the top-K_c entries by |assoc|, and scored on the current batch. Growth stops after $\ell_{\max}$ literals, after L_s consecutive extension depths without improving the best accuracy seen across depths, when the beam is empty, or once accuracy is perfect on $\mathcal{B}_n$. When the batch fills, the builder selects the best candidate across all nonempty depth layers by descending in-sample accuracy, then by |assoc|, and stores it as a `ConjunctionExpert`. The expert is appended to the ensemble with weight zero. If the ensemble exceeds capacity $K_{\max}$, the expert with smallest $|w_k|$ is removed, breaking ties toward the oldest index. There is no hold-out batch and no shared candidate queue across batches.

3.2 Conjunction evaluation

A stored `ConjunctionExpert` lists up to $\ell_{\max}$ signed literals in sorted feature order. Evaluation is a single pass: $f_k(x) = 1$ iff every positive literal sees a one and every negated literal sees a zero. One forward pass evaluates $f_k(x)$ for prediction and SGD.

3.3 Supervision and batching

Batching amortizes conjunction search over B supervision bits. Batch n is

$$\mathcal{B}_n = \{(x_t, s_t)\}_{t \in I_n}, \qquad |I_n| = B,$$

where I_n is the contiguous block of B observations since the previous structural update. Processing $\mathcal{B}_n$ runs the beam search, selects one conjunction by in-sample accuracy, appends $(f, w = 0)$ to the ensemble, and prunes to $K_{\max}$ if needed.

Association score. For a boolean batch column $z_{1:B}$ and supervision labels $s_{1:B}$, let $N = B$, $N_x = \sum_i z_i$, $N_y = \sum_i s_i$, and $N_{xy} = \sum_i z_i s_i$. Define

$$\text{assoc}(z, s) = N\, N_{xy} - N_x N_y.$$

Beam candidates are ranked by descending in-sample accuracy; ties break on $|\text{assoc}(z, s)|$. Layer 0 enumerates both polarities of each non-constant input bit, so negated literals enter the beam without a separate negation node.

3.4 Layered beam extension

Starting from depth 0, the builder repeatedly extends to the next depth until a structural stop triggers:

1. While fewer than L_s consecutive extension depths have failed to improve the best accuracy across depths, the beam is nonempty, depth is below $\ell_{\max} - 1$, and accuracy is not yet perfect on $\mathcal{B}_n$:
   (a) For each parent in depth d, form extensions by AND-ing the parent batch column with each admissible signed literal column from depth 0.
   (b) Deduplicate by `ConjunctionKey`, rank by |assoc|, and keep the top-K_c survivors as depth d+1.

After growth stops, the builder returns the best candidate across all retained depth layers by accuracy, then |assoc|.

3.5 Ensemble pruning

When appending a new expert would exceed $K_{\max}$, remove the stored index with smallest $|w_k|$, breaking ties toward the oldest index. Zero-weight experts can therefore rotate at capacity even when still structurally useful on past batches.

4 Runtime data structures and memory

The online models preallocate all hot-path storage at construction from $(d, B, K_{\max}, K_c, \ell_{\max}, L_s, C)$; extension-buffer widths are derived from (d, K_c) at construction. After `new`, the ensemble `predict`/`update` loop and in-place conjunction batch build perform **zero heap allocation**: no `Vec` growth, hash maps, or per-call scratch vectors on those paths. Standalone batch learning exposes the same builder through `ConjunctionBuildSession`, which owns one `ConjunctionBuildWorkspace`; its `build` method is zero-alloc after session construction. The one-shot `ConjunctionBuilder::build` entry point materializes a `ConjunctionExpert` for tests and scripting.

Persistent model state. `BRegressor` and `BClassifier` wrap a `BooleanEnsemble` with a task-specific head and a fixed `EnsembleWorkspace`. Experts live in `functions: Box<[ConjunctionExpert]>` with capacity $K_{\max}$; active count $F \le K_{\max}$ is tracked in the head. Pruning copies slot j+1 into slot j and clears the trailing slot in place. Each `ConjunctionExpert` stores up to $\ell_{\max}$ signed literals and a live literal count.

Type	Fields	Capacity	Role
`EnsembleConfig`	η, λ, S, B, K_c, $K_{\max}$, $\ell_{\max}$, L_s	fixed	hyperparameters
`RegressionHead`	`intercept`, `weights: Box<[f32]>`, `active`	$K_{\max}$ weights	linear head
`ClassificationHead`	C, `intercepts`, `weights: Box<[f32]>`, `active`	$C{+}K_{\max}C$ floats	softmax head
`ConjunctionExpert`	signed literals, count	$\ell_{\max}$	one expert f_k
`BooleanEnsemble`	config, head, `functions`, `workspace`	$K_{\max}$ slots	online engine

Ensemble workspace (hot path). `EnsembleWorkspace` is allocated once in `BooleanEnsemble::new` and reused across observations:

Buffer	Shape	Use
`function_values`	$K_{\max}$ bool	latest $f_k(x_t)$
`pending_features`	d bool	`predict` pairing
`pending_function_values`	$K_{\max}$ bool	`update` SGD row
`logits`	C f32	classifier head
`batch_features`	Bd bool	column-major partial batch
`batch_signs`	B bool	supervision bits
`build`	`ConjunctionBuildWorkspace`	in-place beam search

Batch columns are exposed to the builder as a flat `SignBatch` view over `batch_features` without allocating column pointer vectors.

Batch-build workspace. Each `ConjunctionBuildWorkspace` holds fixed pools sized by $(d, B, K_c, \ell_{\max})$: precomputed signed-literal columns ($2dB$ bool), $\ell_{\max}$ `DepthLayer` slots (each with K_c beam entries and K_cB column bits), extension and deduplication scratch ($O(K_cd)$ `ExtensionCandidate`), sort indices, and one B-row AND scratch buffer. `build_in_workspace` resets layer lengths and reuses these buffers; the selected `ConjunctionKey` lowers directly into a preallocated expert slot via `ConjunctionExpert::from_key`.

Type	Fields	Size	Lifetime
`ConjunctionBuildSession`	`ConjunctionBuildWorkspace`, config	$O(K_cdB)$	reusable builder
`SignBatch`	columns or flat Bd layout	borrowed	one build
`ConjunctionExpert`	literal array	$O(\ell_{\max})$	one expert

Memory scaling and allocation policy. Ignoring slice headers, resident ensemble memory scales as $4K_{\max}$ bytes for regression weights or $4C(1{+}K_{\max})$ for classification, plus $K_{\max}\,O(\ell_{\max})$ for expert literal storage, plus hot/build workspace $O(K_{\max} + d + C + Bd + K_c\ell_{\max}B + K_cd)$ bytes derived at construction. Structural work peaks once every B updates inside the preallocated `build` pool; per-observation work reuses head scratch without heap growth. Regression tests assert zero allocations after `new` on the ensemble and `ConjunctionBuildSession` hot paths.

5 Online head updates

Expert identity is discrete; head parameters are continuous, so the two are updated separately. For each observation, `predict` evaluates the current experts once; `update` then computes the head-specific supervision bit s_t, appends (x_t, s_t) to the current expert batch, and performs S single-sample gradient steps on that activation row using the task-specific rules above. In both models the newly appended expert at a batch boundary is excluded from SGD on the batch-fill step because it is added only after those updates. There is no gradient through boolean structure: SGD tracks task loss at the sample level, while structure search runs when enough supervision bits have accumulated. After every B new observations, the batch is consumed to grow, select, compact, and append one expert, then cleared.

6 Online learning procedure

Configuration

Linear Learning rate η, decay λ, and S SGD steps per observation.

Batch structure Expert batch size B, beam width K_c, maximum conjunction length $\ell_{\max}$, stale-layer patience L_s, and ensemble capacity $K_{\max}$.

Initialize: empty ensemble, $b \leftarrow 0$, and empty expert-batch buffers.
For each observation (x_t, y_t):

1. Validate the input width and task target.
2. Evaluate each expert $f_k(x_t)$ and form the task prediction ($\hat{y}_t$ or $\hat{c}_t$).
3. Compute the head-specific supervision bit s_t and append (x_t, s_t) to the current expert batch.
4. Apply S single-sample head updates using the current activation row.
5. After B accumulated observations, run conjunction beam search on the batch, select one output, append it with weight zero, prune to $K_{\max}$ if needed, and clear the expert batch.

Cost. Prediction and activation evaluation are linear in the number of experts and their literal counts. Each linear step costs one expert evaluation. Structural work occurs once per expert batch: beam extension scans candidate literal extensions over B rows, then one `ConjunctionExpert` is stored.

7 API and integration boundaries

The `binaml.models` package owns the online models and has no environment dependency: the goal is generic models, decoupled from where the data comes from and how it was produced. The public surfaces are `BRegressor` and `BClassifier`, which implement `predict(features)` and `update(target)` for binary feature vectors of width d. The prediction call retains pairing state; `update` must follow a preceding `predict` and applies the revealed target to that stored step. Expert construction is internal to `update` once a batch fills. Evaluation packages compose models with environments through predict-then-update prequential evaluation. Named benchmarks only compose these independent components.

8 Prequential evaluation protocol

For each emitted sample, an evaluator first records the model prediction, then exposes the target through `update`. Regression benchmarks compute mean squared error from the recorded pre-update predictions; classification benchmarks compute accuracy from the recorded pre-update class predictions. Structure updates occur only after a full expert batch fills inside `update`; residual signs are measured with the current head parameters before the corresponding updates. Hyperparameters, implementation version, and the online protocol are required to interpret any reported trajectory.

9 Evaluation

The benchmark inputs and model configurations are versioned in this paper's benchmark directory.

9.1 Regression

The regression task uses 1,000 observations and seeds $\{0, 1, 2, 3, 4\}$ with 32 input bits, 10 fixed target components $f_k(x)$ sampled as conjunctions of one to 32 literals each, noise standard deviation 0.1, and the same function, DAG, and target-scale settings. Input bits are resampled from the input DAG every step ($p_{\text{sample},x} = 1$); gate bits use partial resampling ($p_{\text{sample},g} = 0.2$). Input, gate, and intercept distributions each resample independently with probability 0.01. Scenario `default.json` uses ten components; the lighter `single_function.json` variant uses one such component with one to 32 literals; `single_function_high_arity.json` uses one component with three to seven literals. Scenario `default_no_drift.json` matches the default regression setup with drift probabilities set to zero.

All models receive the scenario width. Model hyperparameters are versioned in `benchmarks/models.json` and `benchmarks/models_classification.json`. For regression, `BRegressor` uses learning rate $\eta = 3 \times 10^{-2}$, ℓ_2 decay 7.5×10^{-3}, expert batch size $B = 8$, thirty-five single-sample SGD steps, beam width $K_c = 8$, maximum conjunction length $\ell_{\max} = 7$, stale-layer patience $L_s = 4$, and ensemble capacity $K_{\max} = 32$. The linear baseline uses learning rate 5×10^{-3}, decay 1.5×10^{-2}, replay size $B = 1$, three full-batch SGD steps, and uncentered binary inputs. The MLP baseline is a JAX two-layer ReLU network with hidden layers of 32 and 32 units, learning rate 7.5×10^{-2}, $\alpha = 3 \times 10^{-3}$, replay size $B = 1$, thirty-one full-batch SGD steps, and random state zero. Linear and MLP baselines share the same float32 replay SGD scaffold. The last-target baseline predicts zero on the first observation and thereafter repeats the most recently observed target, ignoring the input features.

Tables report the mean ± standard error over five seeds. MSE is the optimized loss; RMSE expresses the same error in target units. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	MSE	RMSE	total s
BRegressor (ours)	0.076 ± 0.016	0.268 ± 0.032	0.076 ± 0.001
Linear SGD (JAX)	0.068 ± 0.012	0.256 ± 0.026	1.336 ± 0.009
MLP SGD (JAX)	0.069 ± 0.014	0.255 ± 0.030	2.303 ± 0.077
Last target	0.078 ± 0.020	0.270 ± 0.037	0.019 ± 0.000

The no-drift variant in `benchmarks/scenarios/default_no_drift.json` uses the same settings as `default.json` with $p_x = p_g = p_b = 0$ and fixed gate bits (no gate resampling).

Model	MSE	RMSE	total s
BRegressor (ours)	0.056 ± 0.018	0.223 ± 0.040	0.076 ± 0.002
Linear SGD (JAX)	0.033 ± 0.009	0.176 ± 0.026	1.235 ± 0.021
MLP SGD (JAX)	0.052 ± 0.015	0.217 ± 0.035	2.503 ± 0.054
Last target	0.077 ± 0.029	0.255 ± 0.054	0.019 ± 0.001

A lighter variant in `benchmarks/scenarios/single_function.json` uses the same settings with one fixed conjunction f_0 instead of ten components.

Model	MSE	RMSE	total s
BRegressor (ours)	0.041 ± 0.006	0.201 ± 0.014	0.072 ± 0.003
Linear SGD (JAX)	0.050 ± 0.007	0.220 ± 0.017	1.232 ± 0.036
MLP SGD (JAX)	0.034 ± 0.006	0.183 ± 0.014	2.471 ± 0.027
Last target	0.034 ± 0.006	0.181 ± 0.016	0.018 ± 0.000

The high-arity variant in `benchmarks/scenarios/single_function_high_arity.json` uses the same settings with one fixed conjunction f_0 whose clause has three to seven literals.

Model	MSE	RMSE	total s
BRegressor (ours)	0.047 ± 0.006	0.215 ± 0.015	0.065 ± 0.002
Linear SGD (JAX)	0.055 ± 0.006	0.234 ± 0.012	1.183 ± 0.020
MLP SGD (JAX)	0.040 ± 0.004	0.199 ± 0.011	2.536 ± 0.054
Last target	0.040 ± 0.005	0.197 ± 0.012	0.018 ± 0.000

9.2 Classification

The classification task uses the same input width, component count, noise standard deviation, drift probabilities, and f_k clause settings as the regression default, with ten classes and per-class gates, weights, and intercepts. Binary classification is the same generator with `n_classes=2`; a dedicated scenario pins one f_0 and fixed class head weights. Scenario and model entries are versioned in `benchmarks/scenarios/default_multiclass.json`, `benchmarks/scenarios/default_multiclass_no_drift.j` `benchmarks/scenarios/single_function_multiclass.json`, `benchmarks/scenarios/single_function_high_` `benchmarks/scenarios/binary.json`, and `benchmarks/models_classification.json`. `BClassifier` uses learning rate $\eta = 9 \times 10^{-2}$, ℓ_2 decay 5×10^{-4}, expert batch size $B = 8$, thirty-nine single-sample SGD steps, beam width $K_c = 8$, maximum conjunction length $\ell_{\max} = 7$, stale-layer patience $L_s = 4$, and ensemble capacity $K_{\max} = 40$. The linear classifier uses learning rate 5×10^{-3}, no weight decay, replay size $B = 1$, and seven SGD steps; the MLP classifier baseline uses three ReLU hidden layers of 50, 32, and 16 units, learning rate 5×10^{-3}, $\alpha = 2.5 \times 10^{-4}$, replay size $B = 1$, and nine SGD steps, both with softmax cross-entropy. The last-target baseline uses the same rule as in regression, predicting class zero initially. Reported comparisons use prequential accuracy over the same seed set after optional warm-up exclusion.

Tables report the mean ± standard error over five seeds. Accuracy is the fraction of correct pre-update class predictions on the 900 post-warm-up observations. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	accuracy	total s
BClassifier (ours)	0.621 ± 0.026	0.096 ± 0.001
Linear SGD (JAX)	0.652 ± 0.022	1.561 ± 0.168
MLP SGD (JAX)	0.646 ± 0.020	1.967 ± 0.067
Last target	0.623 ± 0.026	0.014 ± 0.000

The no-drift variant in `benchmarks/scenarios/default_multiclass_no_drift.json` uses the same settings as `default_multiclass.json` with $p_x = p_g = p_b = 0$ and fixed gate bits (no gate resampling).

Model	accuracy	total s
BClassifier (ours)	0.625 ± 0.132	0.088 ± 0.003
Linear SGD (JAX)	0.741 ± 0.102	1.015 ± 0.016
MLP SGD (JAX)	0.688 ± 0.115	2.073 ± 0.031
Last target	0.590 ± 0.142	0.014 ± 0.000

The single-function variant in `benchmarks/scenarios/single_function_multiclass.json` uses one fixed conjunction f_0 with the same multiclass head sampling as the default.

Model	accuracy	total s
BClassifier (ours)	0.664 ± 0.020	0.087 ± 0.001
Linear SGD (JAX)	0.676 ± 0.022	1.026 ± 0.029
MLP SGD (JAX)	0.680 ± 0.021	2.057 ± 0.039
Last target	0.668 ± 0.022	0.014 ± 0.000

The high-arity variant in `benchmarks/scenarios/single_function_high_arity_multiclass.json` uses one fixed conjunction f_0 whose clause has three to seven literals.

Model	accuracy	total s
BClassifier (ours)	0.648 ± 0.023	0.090 ± 0.002
Linear SGD (JAX)	0.674 ± 0.023	1.065 ± 0.016
MLP SGD (JAX)	0.676 ± 0.022	2.002 ± 0.046
Last target	0.658 ± 0.024	0.014 ± 0.000

10 Limitations and scope

This document specifies the ensemble models used by Binaml. It does not claim optimality of residual-sign learning or in-sample expert selection. Ensemble churn at capacity can drop zero-weight experts that remain useful on past batches; in-sample output selection trades hold-out promotion for simpler batch boundaries. The reported regression comparison is limited to the versioned synthetic task, fixed configurations, and reproducible baseline runs described above.